Základy počítačové grafiky
Přednáška 6

Martin Němec

VŠB-TU Ostrava

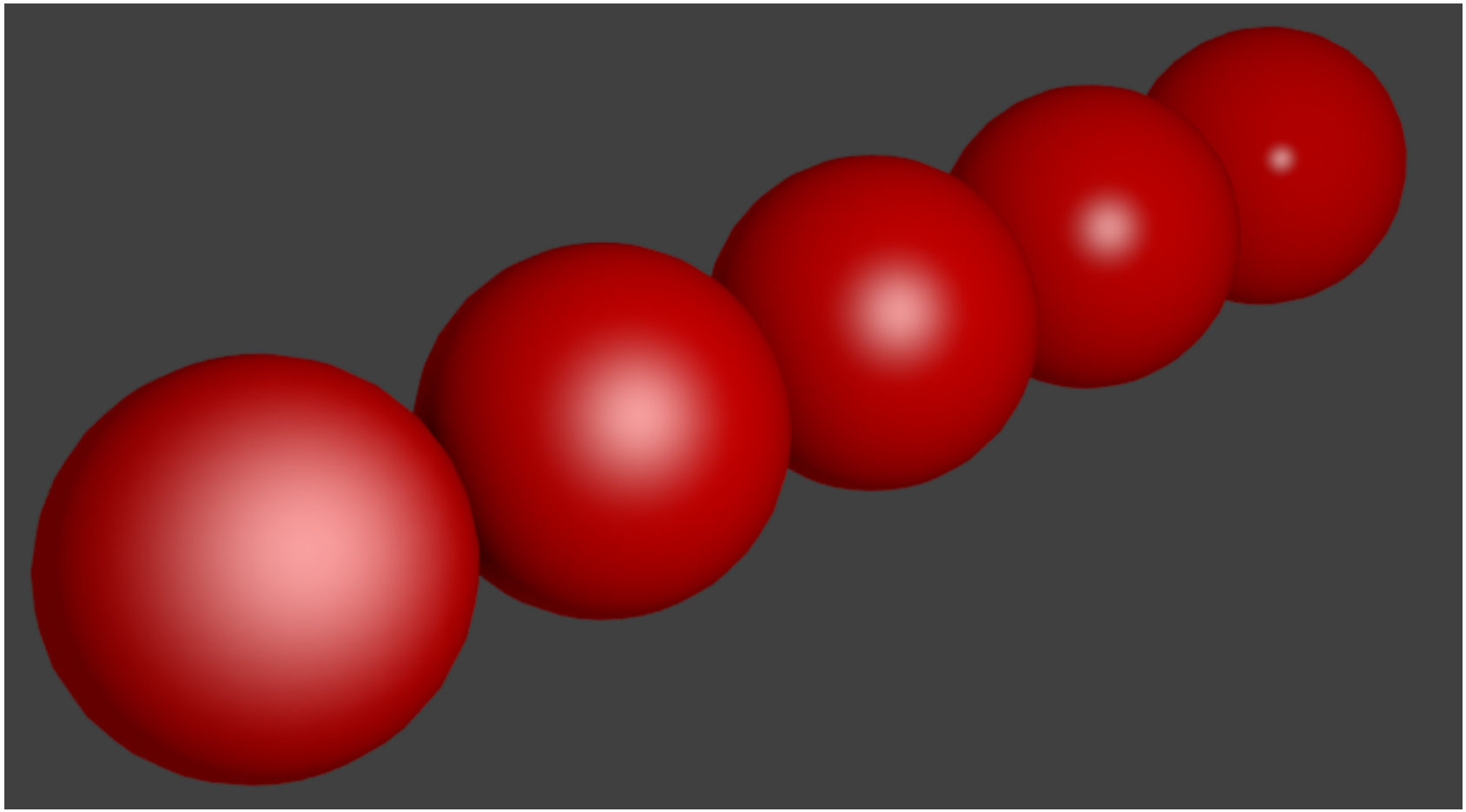
2026

VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY | KATEDRA INFORMATIKY

Úvod

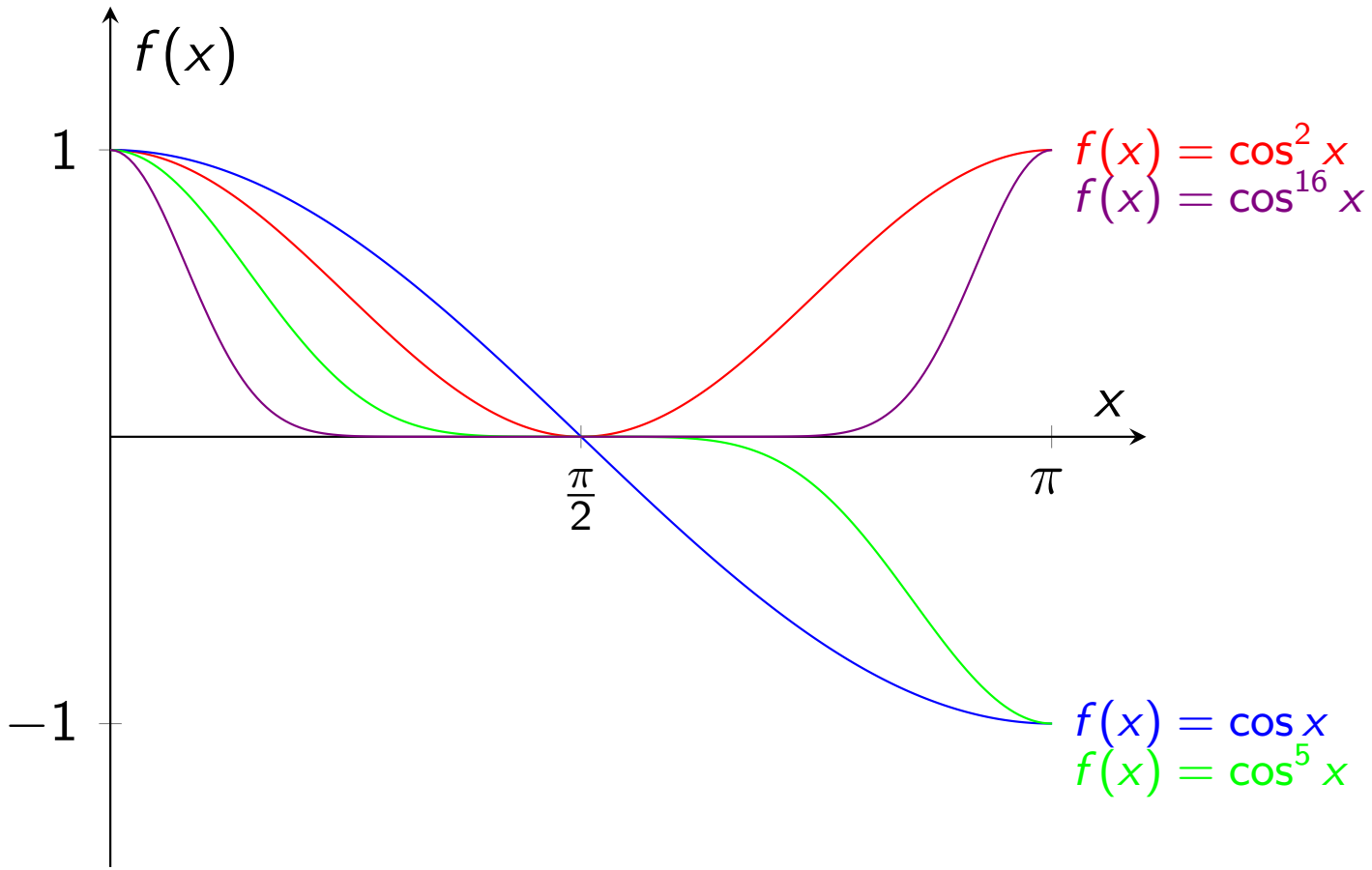
- Využívejte cvičení a přednášky aktivně, nejprve se snažte pochopit a poté naprogramovat.
- Snažte se provádět menší změny, aby je bylo možné otestovat.
- Používejte verzovací systémy (https://gitlab.vsb.cz/).
- Nebojte se testovat různé věci (ShaderProgramFactory, ModelManager, SceneBuilder atd.).

Shininess constant



Shininess constant

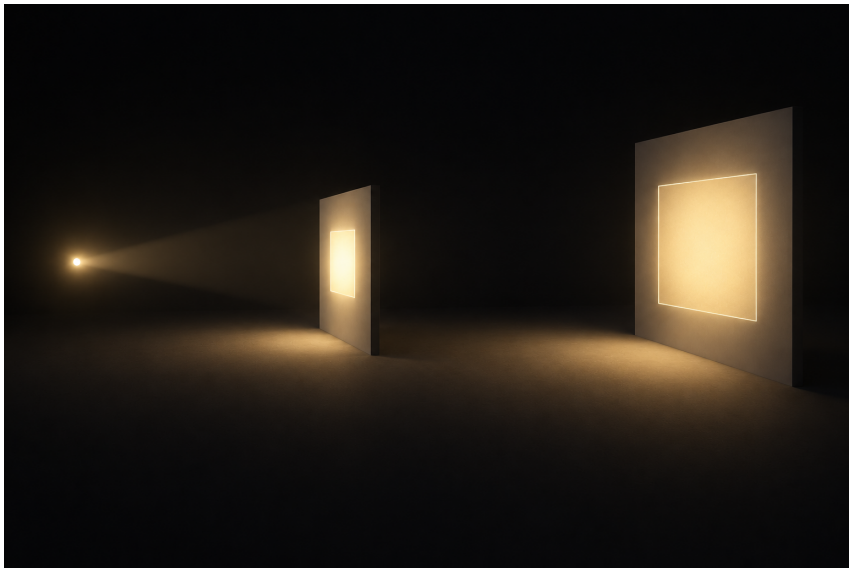
Ukázka změny průběhu funkce kosinus s různými koeficienty.



Light attenuation

Ve vakuu se světlo při šíření ideálně neztrácí absorpcí, ale rozprostírá se na stále větší plochu. Plocha dopadu roste se vzdáleností. Při 2x větší vzdáleností je plocha 4x větší.

$$f_{\mathrm{att}}(d) = \frac{1}{d^2}$$



Light attenuation

V modelu osvětlení OpenGL se útum popisuje vztahem

$$f_{\text{att}}(d) = \frac{1}{k_c + k_l d + k_q d^2}$$

kde d je vzdálenost od světelného zdroje a k_c, k_l, k_q jsou konstantní, lineární a kvadratický koeficient útlumu.
Hodnoty koeficientů nastavujeme podle požadovaného vzhledu scény. Vše ovlivňuje měřítko, typ scény, osvětlovací model apod.

```
//fixni pipelina
glLightf(GL_LIGHT0, GL_CONSTANT_ATTENUATION, 1.0f);
glLightf(GL_LIGHT0, GL_LINEAR_ATTENUATION, 0.09f);
glLightf(GL_LIGHT0, GL_QUADRATIC_ATTENUATION, 0.032f);
```

Light attenuation

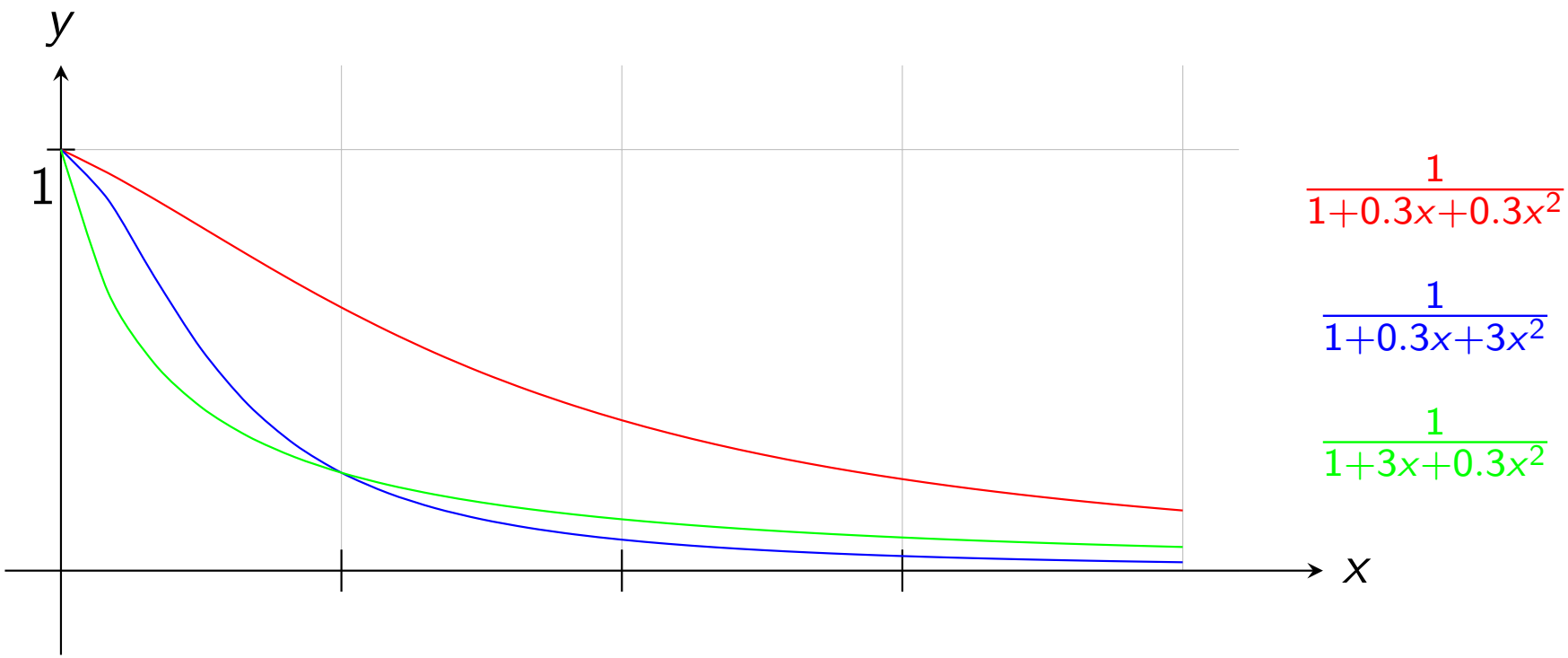
Hodnoty nejsou obvykle předem pevně dané, upravujeme je podle požadavků na výslednou scénu.

$$f_{att}(d) = \frac{1}{k_c + k_l d + k_q d^2}$$

```
// Point light attenuation calculation
float attenuation(float d, float c, float l, float q)
{
    // d = distance from the light to the fragment
    // c = constant attenuation factor
    // l = linear attenuation factor
    // q = quadratic attenuation factor
    float att=1.0/ (c + l*d + q*d*d)
    return clamp(att , 0.0 , 1.0);
}
float att = attenuation(dist, 1.0, 0.09, 0.032);
```

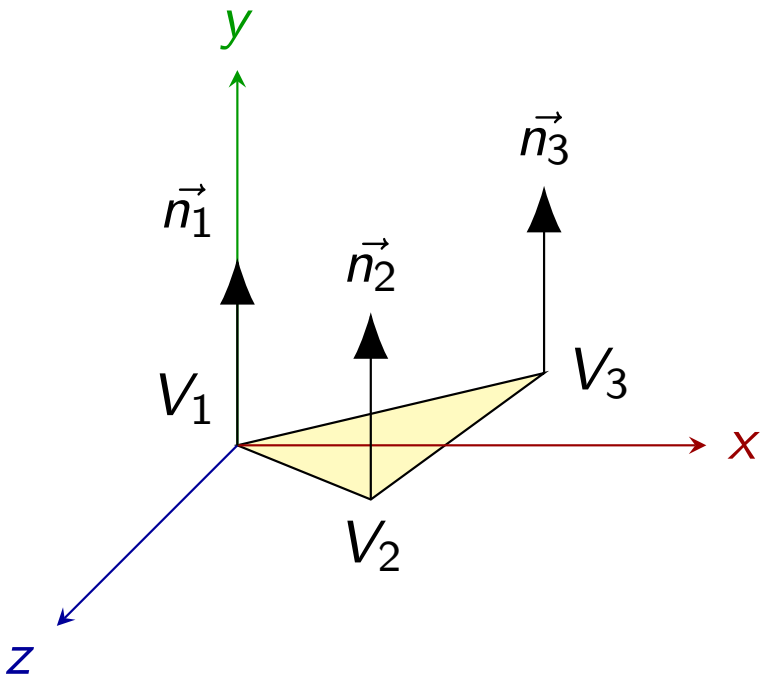
Light attenuation

$$f_{att}(d) = \frac{1}{1 + k_l d + k_q d^2}$$



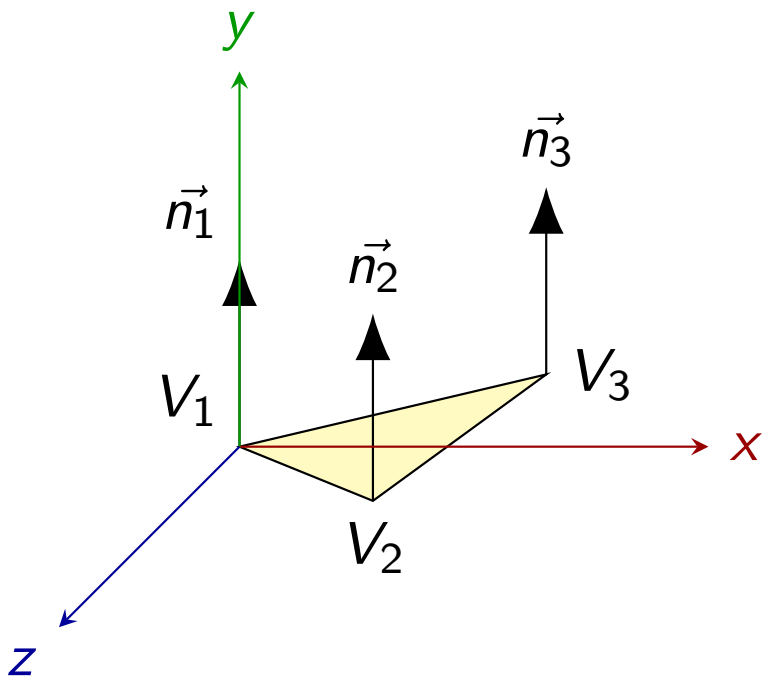
Transformace objektů

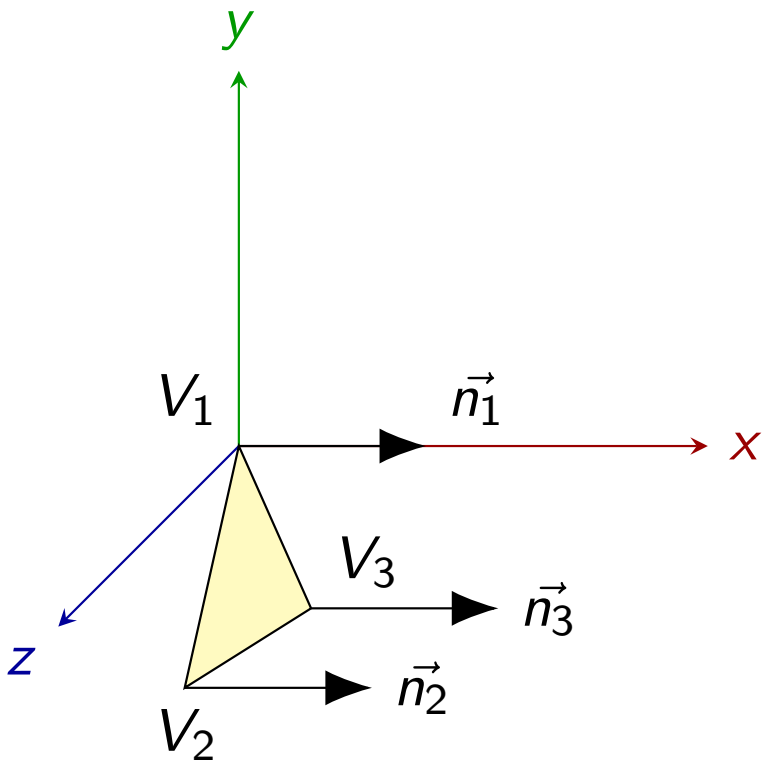
Mějme trojúhelník o vrcholech V_1, V_2 a V_3, který má v každém vrcholu normálový vektor $\vec{n_1}, \vec{n_2}$ a $\vec{n_3}$. Převeďte objekt z local space do world space. Jak převedeme vrcholy a jak normály?



Rotace tělesa

Proveďte rotaci o 90° kolem osy z.

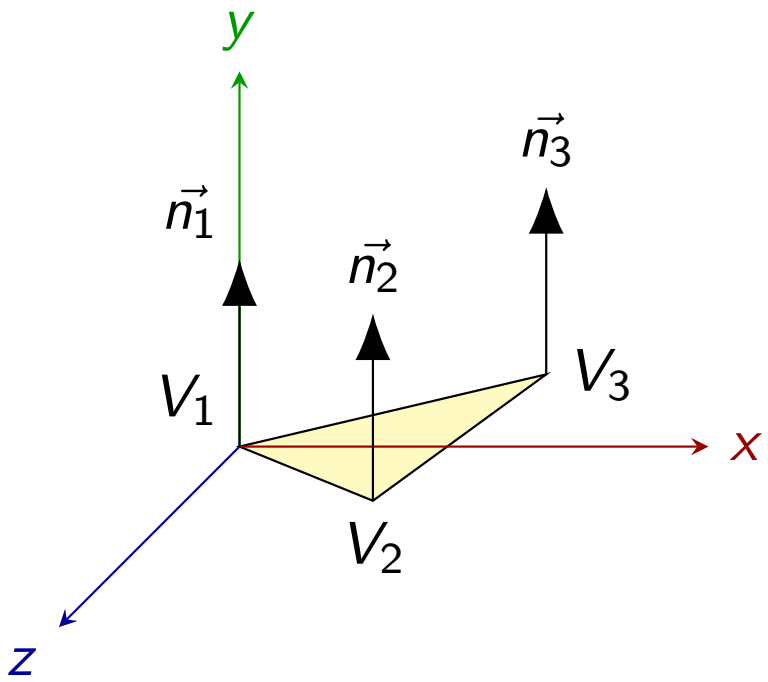


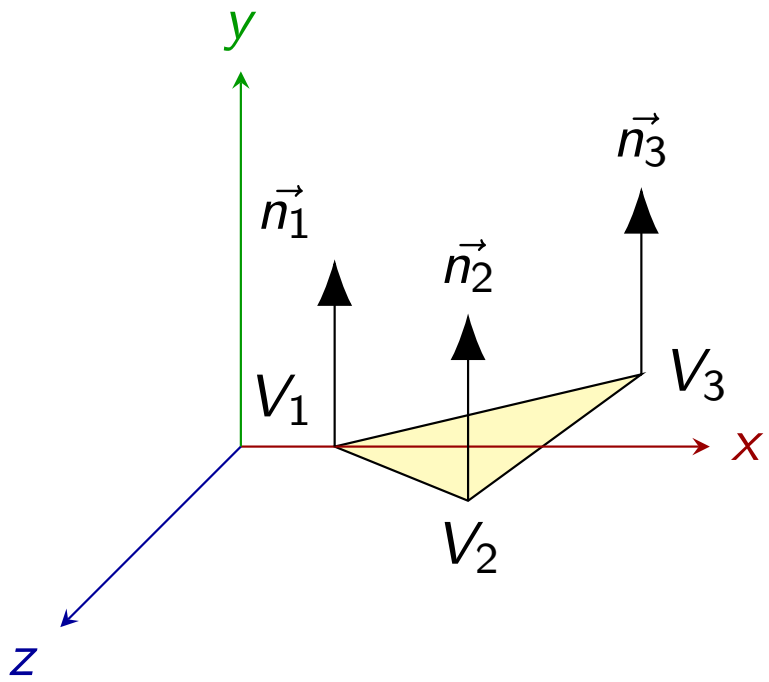


Rotaci chceme na normálové vektory použít.

Translace normálových vektorů

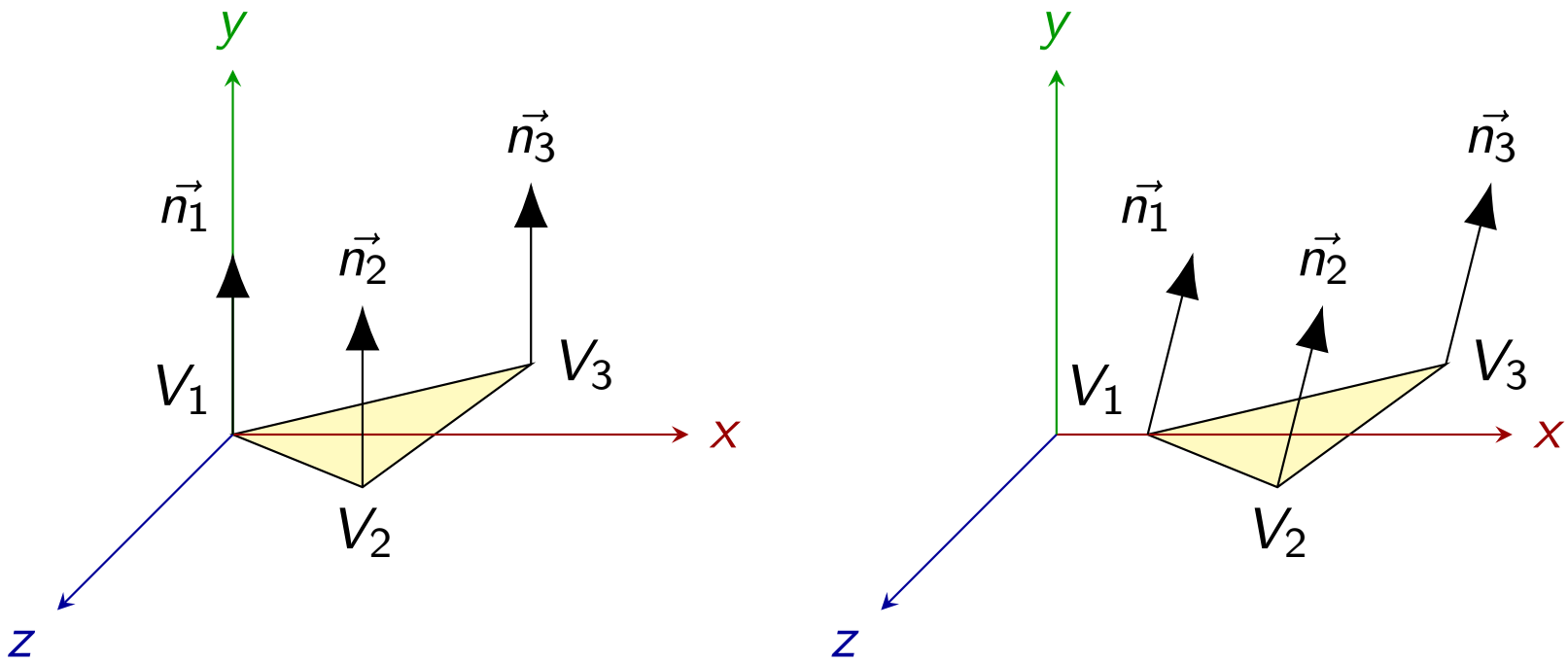
Proveďte translaci ($T(0.2, 0, 0)$). Můžeme translaci na normály použít stejně jako rotaci?





Translace normálových vektorů

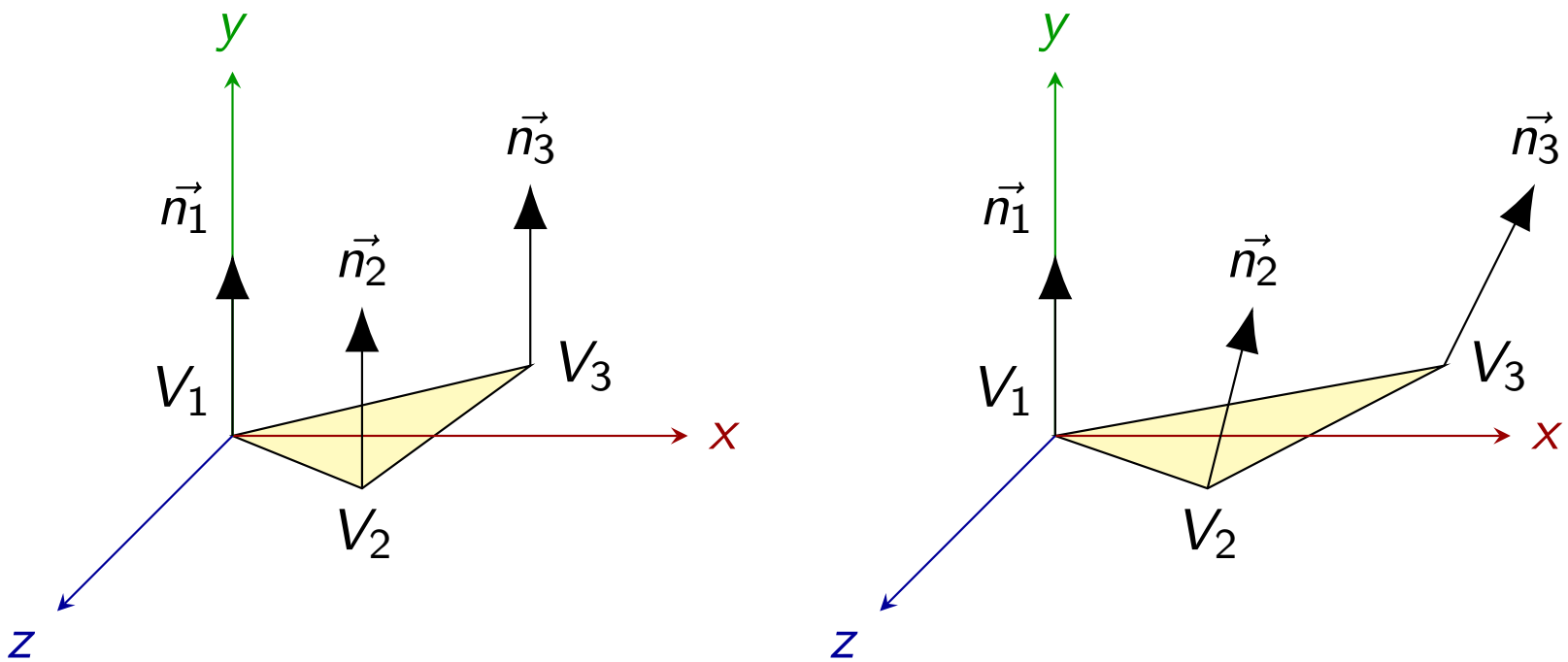
Proveďte translaci ($T(0.2, 0, 0)$). Můžeme translaci na normály použít stejně jako rotaci?



Translaci na normálové vektory použít nesmíme.

Scale normálových vektorů

Obdobně jako u translace vzniká problém při použití neuniformní změny měřítka (různá měřítka na osách).



Scale na normálové vektory použít nesmíme.

Převod světla a kamery do Object Space

Můžeme převést pozici světla (bod) do souřadnicového systému modelu (z globálního do lokálního).
Můžeme využít inverzní matici $\mathbf{M}\mathbf{M}^{-1} = \mathbf{M}^{-1}\mathbf{M} = \mathbf{I}$.

$$V_1' = \mathbf{M} V_1$$

Potom platí:

$$\mathbf{M}^{-1} V_1' = \mathbf{M}^{-1}\mathbf{M} V_1$$

$$\mathbf{M}^{-1} V_1' = \mathbf{I} V_1$$

$$V_1 = \mathbf{M}^{-1} V_1'$$

Tečný vektor

Tečný vektor $\vec{t}$ je určen dvěma vrcholy $\vec{t} = V_2 - V_1$.
Při transformaci platí:

$$\vec{t'} = \mathbf{M}\vec{t} = \mathbf{M}(V_2 - V_1) = \mathbf{M}V_2 - \mathbf{M}V_1.$$

Transformovaný tečný vektor je:

$$\vec{t'} = V_2' - V_1'.$$

Tečný vektor transformujeme lineární částí modelové matice (matice 3×3 zahrnující rotaci a změnu měřítka).
Normálový vektor $\vec{n}$ je kolmý k povrchu a nelze jej obecně transformovat stejnou maticí jako tečný vektor, protože při neuniformním měřítku by se nezachovala kolmost.

Normálová matice

Odvození normálové matice pro transformaci normálových vektorů.

$$\begin{aligned} dot(\vec{n}, \vec{t}) &= 0 \\ dot(\vec{n'}, \vec{t'}) &= 0 \end{aligned}$$

$$\begin{aligned} \vec{n}^T \vec{t} &= 0 \\ \vec{n'}^T \vec{t'} &= 0 \end{aligned}$$

$$\begin{aligned} \vec{t'} &= \mathbf{M}\vec{t} \\ \vec{n'} &= \mathbf{N}\vec{n} \end{aligned}$$

$$\begin{aligned} dot(\vec{n'}, \vec{t'}) &= 0 \\ \vec{n'}^T \vec{t'} &= 0 \\ (\mathbf{N}\vec{n})^T (\mathbf{M}\vec{t}) &= 0 \end{aligned}$$

Normálová matice

$$\begin{aligned} \vec{t'} &= \mathbf{M}\vec{t} \\ \vec{n'} &= \mathbf{N}\vec{n} \end{aligned}$$

$$\begin{aligned} \vec{n'}^T \vec{t'} &= 0 \\ (\mathbf{N}\vec{n})^T(\mathbf{M}\vec{t}) &= 0 \end{aligned}$$

Protože transpozice součinu matic je součin transponovaných matic v opačném pořadí $(A \cdot B)^T = B^T \cdot A^T$, dostaneme:

$$\vec{n}^T \mathbf{N}^T \mathbf{M} \vec{t} = 0$$

A protože $\vec{n}^T \vec{t} = 0$, pak musí platit

$$\mathbf{N}^T \mathbf{M} = \mathbf{I}$$

a odtud

$$\mathbf{N} = (\mathbf{M}^{-1})^T$$

Vertex shader

```
void main(void)
{
  worldPos = modelMatrix * vec4(vp,1.0);
  //normalova matice
  normalMatrix = transpose(inverse(mat3(modelMatrix)));
  worldNormal = normalize(normalMatrix * normal);

}
```

Počítat inverzní matici je možné jak na CPU tak i GPU.

```
glm::mat3 N=glm::transpose(glm::inverse(glm::mat3(M)));
```

K čemu nám může být toto ?

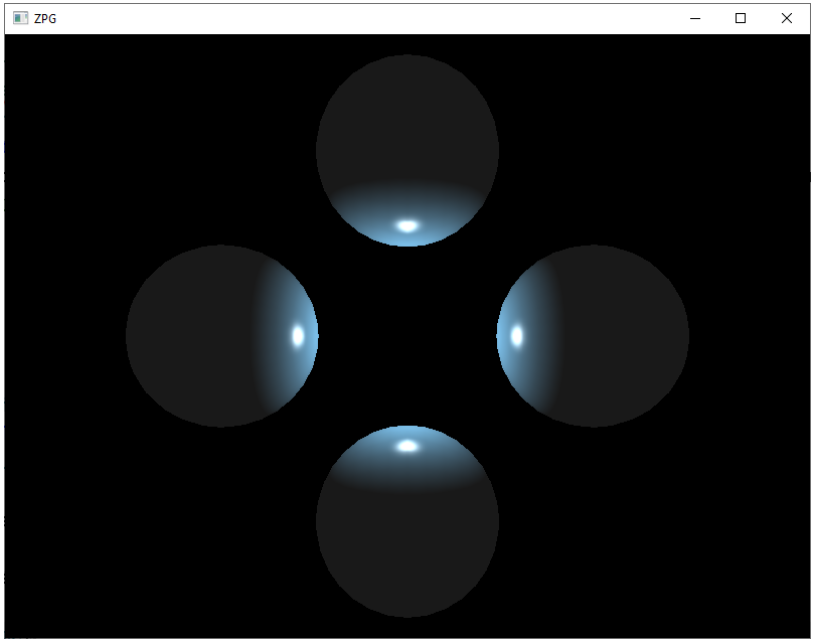
```
glm::mat3 N=glm::transpose(glm::inverse(glm::mat3(VM)));
```

Testování 1

Světlo (S=[0,0,0], h=32) , čtyři kulové plochy souměrně rozmístěné kolem světla na jednotlivých osách a kamera nad objekty.

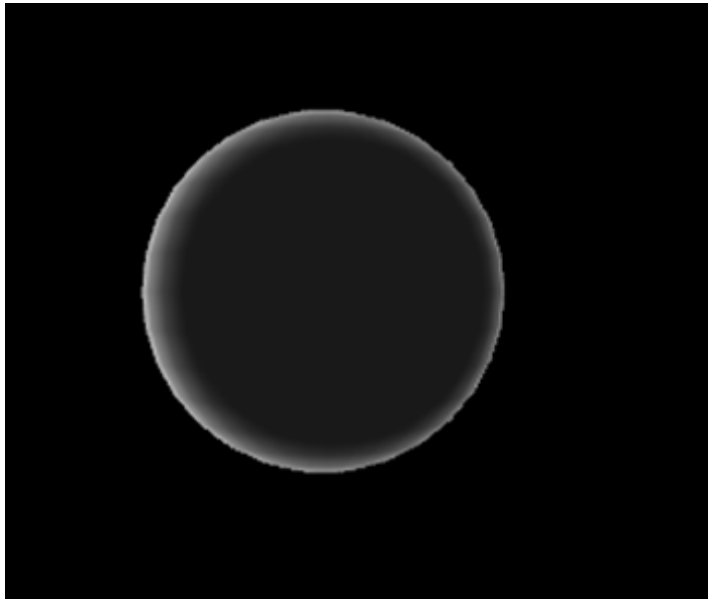
Kontrola správnosti Phongova osvětlovacího modelu (správně ambientní, difuzní i zrcadlová složka.).

Problémy se správným výpočtem (příp. s transformaci normály).



Testování 2

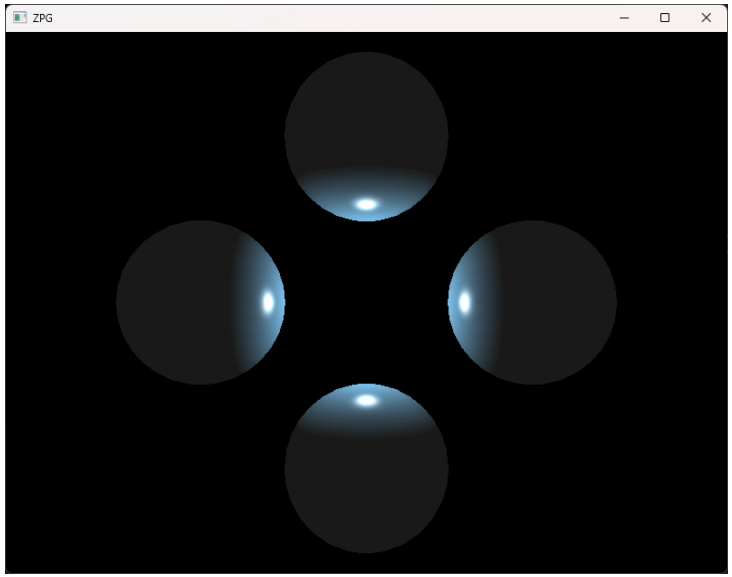
Světlo na špatné straně. Kulová plocha mezi světlem a kamerou, světlo má h=1. Přesvit na druhou stranu.

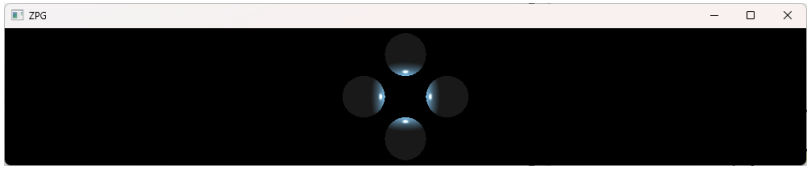


```
// light source on the wrong side?
if (dot(normalDirection, lightDirection) < 0.0) {
    specularReflection = vec4(0.0, 0.0, 0.0, 0.0);
}
```

Testování 3

Test na více modelů ve scéně s více shadery (min. konstantní, lambert, phong a blinn). Při změně obrazovky se nesmějí tělesa deformovat (glViewport a perspective).





Testování 4

Určete, jakou transformaci popisuje následující modelová matice. Prakticky si ji ověřte ve své aplikaci.

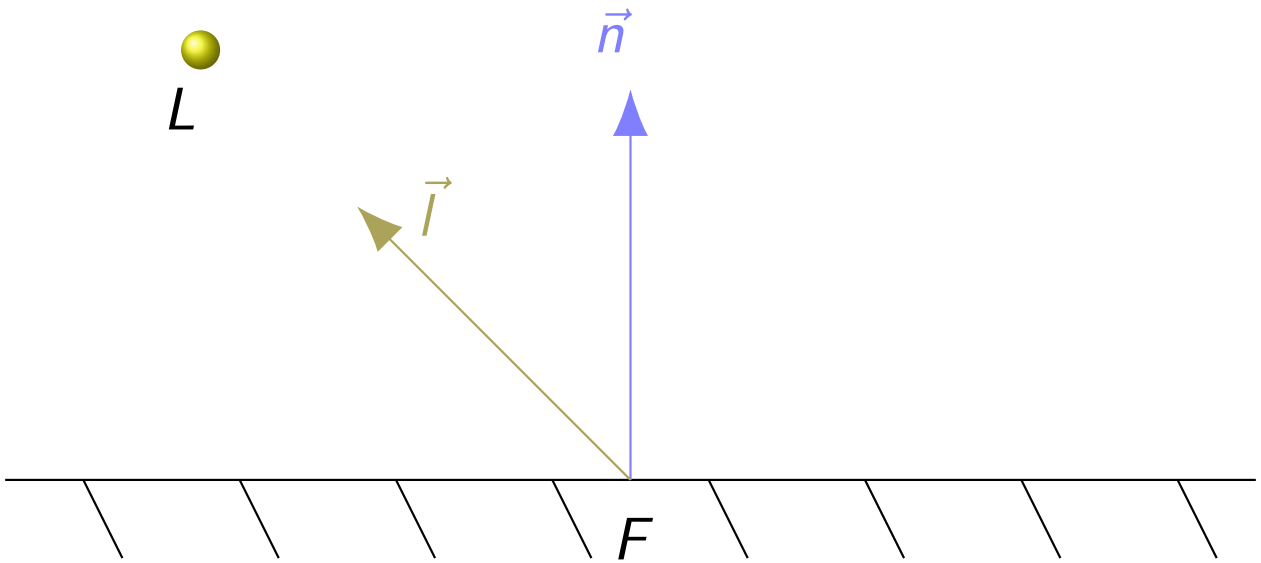
$$modelMatrix = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 5 \end{bmatrix}$$

- Jedná se o posun, rotaci, změnu měřítka nebo jinou transformaci?
- Jaký bude výsledný stav objektu po aplikaci této matice?

```
glm::mat4 modelMatrix = glm::mat4(
    1.0f, 0.0f, 0.0f, 0.0f,
    0.0f, 1.0f, 0.0f, 0.0f,
    0.0f, 0.0f, 1.0f, 0.0f,
    0.0f, 0.0f, 0.0f, 5.0f
);
```

Prakticky

Vypočtěte výslednou intenzitu difúzní složky Phongova osvětlovacího modelu I_d (Lambert), pokud se jedná o bodové světlo bílé barvy na pozici $L = [-2, 2, 0, 2]$, pro aktuálně zkoumaný fragment, který je na pozici $F = [0, 0, 0, 1]$ s normálou $\hat{n} = (0, 1, 0)$.



Prakticky

Vypočtěte I_d (Lambert), bodové světlo bílé barvy na pozici $L = [-2, 2, 0, 2]$, fragment $F = [0, 0, 0, 1]$ s normálou $\hat{n} = (0, 1, 0)$.

Nejprve si převedeme body z homogenních souřadnic na kartézské

$$L_k = [-\frac{2}{2}, \frac{2}{2}, \frac{0}{2}] = [-1, 1, 0]$$
$$F_k = [\frac{0}{1}, \frac{0}{1}, \frac{0}{1}] = [0, 0, 0]$$

Vypočteme jednotkový vektor z aktuálního fragmentu ke světlu $\vec{l} = \vec{FL} = L - F = (-1, 1, 0)$ a převedeme ho na jednotkový

$$\|\vec{l}\| = \sqrt{-1^2 + 1^2 + 0^2} = \sqrt{2}$$
$$\hat{l} = (\frac{-1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, \frac{0}{\sqrt{2}}) = (\frac{-1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, 0)$$

Prakticky

Vypočtěte I_d (Lambert), bodové světlo bílé barvy na pozici $L = [-2, 2, 0, 2]$, fragment $F = [0, 0, 0, 1]$ s normálou $\hat{n} = (0, 1, 0)$.

Vypočteme jednotkový vektor z aktuálního fragmentu ke světlu $\vec{l} = \vec{FL} = L - F = (-1, 1, 0)$ a převedeme ho na jednotkový

$$\|\vec{l}\| = \sqrt{-1^2 + 1^2 + 0^2} = \sqrt{2}$$

$$\hat{l} = (\frac{-1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, \frac{0}{\sqrt{2}}) = (\frac{-1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, 0)$$

Vypočteme skalární součin jednotkového vektoru $\hat{n}$ a jednotkového vektoru $\hat{l}$

$$\langle \hat{n}, \hat{l} \rangle = \frac{-1}{\sqrt{2}} \cdot 0 + \frac{1}{\sqrt{2}} \cdot 1 + 0 \cdot 0$$

$$\langle \hat{n}, \hat{l} \rangle = \frac{1}{\sqrt{2}} \approx 0.7071$$

Prakticky

Vypočtěte I_d (Lambert), bodové světlo bílé barvy na pozici $L = [-2, 2, 0, 2]$, fragment $F = [0, 0, 0, 1]$ s normálou $\hat{n} = (0, 1, 0)$.

Vypočteme skalární součin jednotkového vektoru $\hat{n}$ a jednotkového vektoru $\hat{l}$

$$\langle \hat{n}, \hat{l} \rangle = \frac{-1}{\sqrt{2}} \cdot 0 + \frac{1}{\sqrt{2}} \cdot 1 + 0 \cdot 0$$
$$\langle \hat{n}, \hat{l} \rangle = \frac{1}{\sqrt{2}} \approx 0.7071$$

Phong reflection model (jako $I_d \cdot r_d$ použijeme $(1, 1, 1)$)

$$I_{diffuse} = I_d \cdot r_d \max(0, (\hat{n} \cdot \hat{l}))$$
$$I_{diffuse} = (1, 1, 1) \cdot \frac{1}{\sqrt{2}} = (\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}) \approx (0.7071, 0.7071, 0.7071)$$

glViewport()

Převádí objekty z NDC (Normal Device Coordinates) do souřadnic okna (Window Coordinates). Definuje obdélník pro vykreslení v okně.
glViewport(GLint x, GLint y, GLsizei width, GLsizei height)

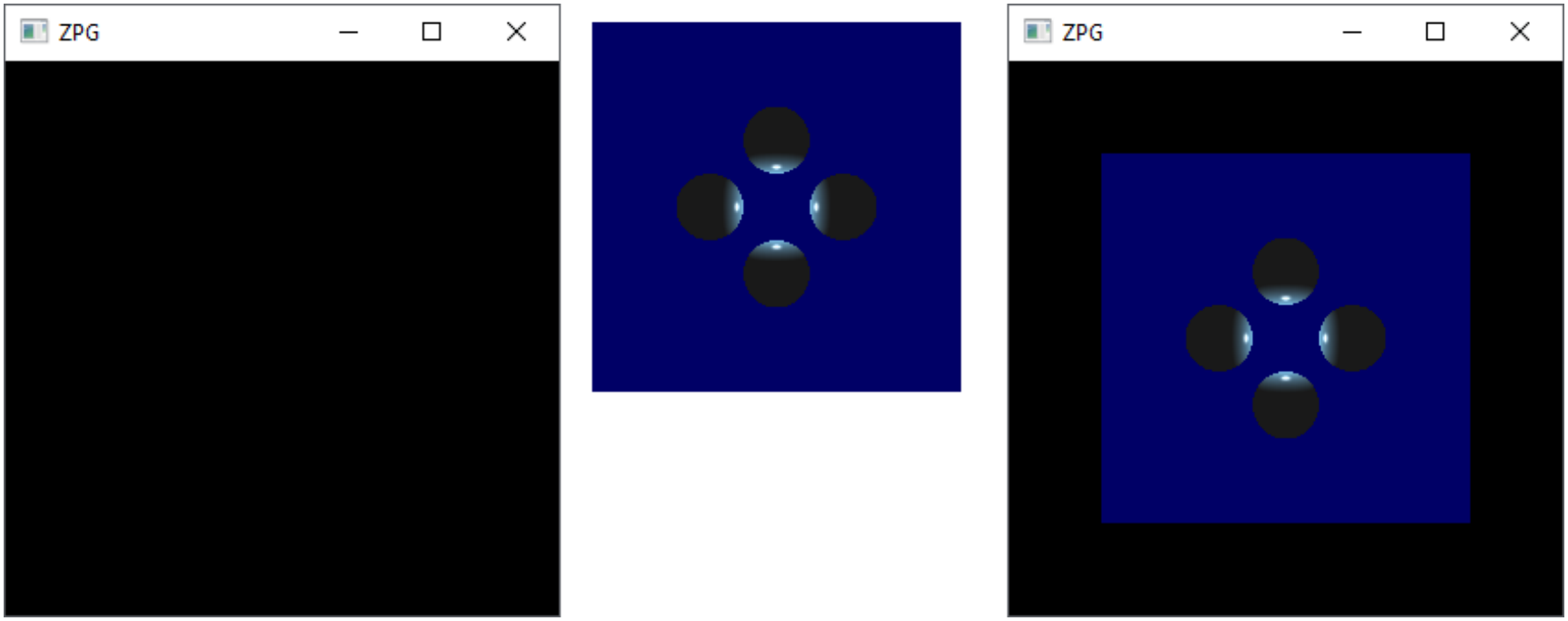
- x, y - pozice levého dolního rohu;
- *width*, *height* - šířka a výška vykreslovaného obdélníku.

$$x_w = (x_{nd} + 1)\left(\frac{width}{2}\right) + x\,,$$

$$y_w = (y_{nd} + 1)\left(\frac{height}{2}\right) + y\,.$$

Jaké souřadnice bude mít bod $A = [-0.5, 1]$, pokud ho převedeme z NDC do okna s následujícím nastavením $(0, 0, 800, 600)$?

glViewport()



glViewport(50, 50, width-100, height-100);

Deklarace a definice v C++

Deklarace – říká, že třída Cat má metodu makeSound(), ale tělo ještě není známé (.h nebo .hpp soubory).

```
class Cat {
public:
    void makeSound(); //method declaration
};
```

Definice – obsahuje těla jednotlivých metod třídy, tedy její implementaci (soubory .cpp).

```
void Cat::makeSound() {
    std::cout << "says:Meow!\n"; //method definition
}
```

Závěr

Dotazy?